Gestión de Configuración de Software (SCM)

Introducción y propósito de la Gestión de Configuración de Software

- Propósito y función
  - Importancia de SCM en el ciclo de vida: Asegura que cambios sean organizados, controlados y trazables desde desarrollo hasta producción.
  - Mejora de la calidad y colaboración: Facilita revisiones, reduce errores y mejora la comunicación entre equipos.
  - Alcance y aplicación práctica: Aplica al código, documentación, entornos y artefactos de construcción para mantener consistencia.

Objetivos principales de la Gestión de Configuración (SCM)

- Propósito y función
  - Control de cambios y aprobaciones: Garantiza que cada modificación esté autorizada, registrada y rastreable con historial completo.
  - Gestión de versiones y reversión: Permite mantener múltiples versiones y restaurar estados anteriores ante fallos o regresiones.
  - Protección de integridad y trazabilidad: Mantiene coherencia con requisitos, auditorías y vinculación entre artefactos y cambios.

Componentes clave de SCM y responsabilidades

- Propósito y función
  - Identificación de ítems de configuración: Determina qué elementos (codigo, docs, scripts, entornos) se gestionan como ítems controlados.
  - Control de configuración y procesos formales: Define procedimientos para solicitar, aprobar, implementar y registrar cambios.
  - Auditoría, validación y gestión de entornos: Verifica cumplimiento de estándares y asegura consistencia entre desarrollo, pruebas y producción.

Herramientas comunes y su rol en SCM

- Propósito y función
  - Git, Subversion y Mercurial (VCS): Control de versiones que proporcionan historial, ramas y mecanismos de fusión para colaboración.
  - Jenkins, CI/CD y automatización: Automatizan compilaciones, pruebas y despliegues integrados con repositorios SCM para entregas continuas.
  - Docker, Ansible, Puppet/Chef (Infraestructura como Código): Gestionan entornos reproducibles y configuraciones consistentes en múltiples capas.

Gestión de cambios: procesos y prácticas

- Propósito y función
  - Flujo de gestión de cambios y roles: Solicitud (RFC), evaluación, aprobación, implementación y cierre con responsables claramente definidos.
  - Resolución de conflictos y coordinación de desarrolladores: Centraliza y coordina esfuerzos para evitar sobrescrituras y problemas de integración.
  - Trazabilidad desde requerimiento a artefacto: Vincula tickets, commits, builds y releases para seguimiento completo del cambio.

Ejemplo práctico y beneficios concretos de SCM

- Propósito y función
  - Caso práctico: implementación exitosa: Implementación con Git + CI/CD + contenedores que redujo errores y aceleró despliegues en un proyecto real.
  - Beneficios operativos y organizativos: Previene pérdida de trabajo, soporta continuidad operativa, mejora calidad y facilita colaboración paralela.
  - Impacto en agilidad y recuperación: Versionado documentado y entornos reproducibles permiten despliegues rápidos y recuperación ante incidentes.